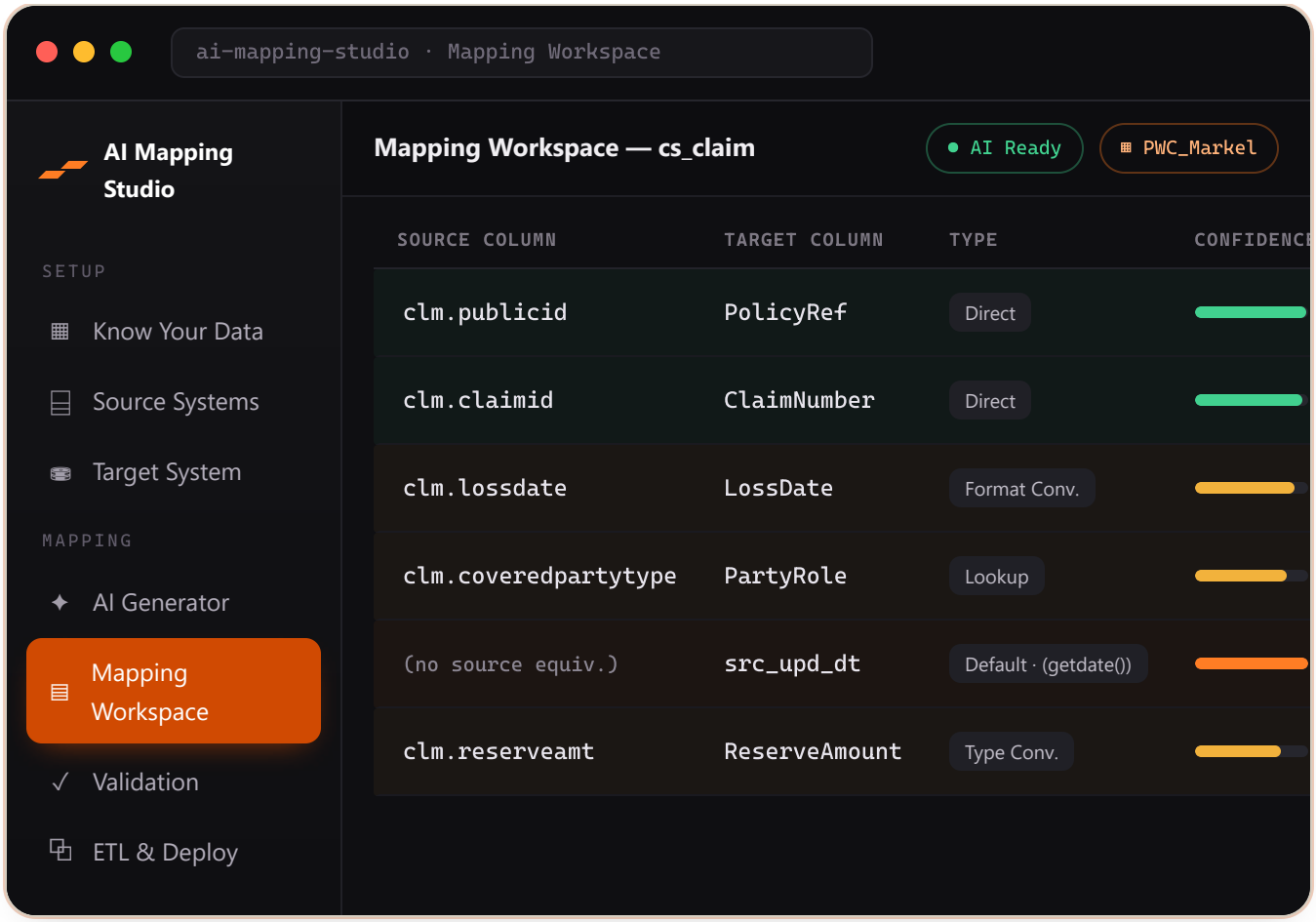
# AI Mapping Studio

PwC · Data Conversion

Executive Brief · Working software · Insurance · Guidewire-inspired

# The mapping layer of a data conversion, drafted by AI — signed off by people.

AI Mapping Studio turns the slowest, most error-prone part of an enterprise data conversion — deciding how every target column is populated from a legacy source — into an **AI-generated draft** that analysts review, correct, validate, and ship as **runnable SQL**.




**Mapping Workspace.** Every target column gets an AI-proposed source, transformation rule and a 0–100 confidence — analysts approve, edit, or regenerate, with row colour showing review status at a glance.

| | |
|---|---|
| **13**<br>AI-assisted operations | **Multi**<br>Tenant & data-isolated |
| **38**<br>API endpoints | **Chat**<br>Ask your own documents |
| **0**<br>Build steps to run | |

THE PROBLEM

# A conversion lives or dies on its mapping.

Every conversion needs a column-level Source-to-Target Mapping: for each target field, which source column feeds it, what transformation applies, and how keys and joins line up. Across hundreds of tables this is slow, manual, and easy to get wrong. AI Mapping Studio proposes the mapping, then keeps a human firmly in control of approving it.

INGEST

### Bring in any schema

Connect a live SQL Server, or upload a data dictionary, DDL, or spreadsheet — extracted automatically, with instant deterministic parsing for SQL and structured Excel.

GENERATE

### AI drafts the mapping

Claude proposes each column's source, transformation rule, business rule, null handling, a 0–100 confidence, and the SQL JOIN that assembles each target entity.

SHIP

### Review → validate → SQL

Approve, edit, or regenerate in the workspace; run validation rules; generate ETL stored procedures and deploy to SQL Server with an AI fix step that a person signs off.

PRODUCT TOUR

# See it working.

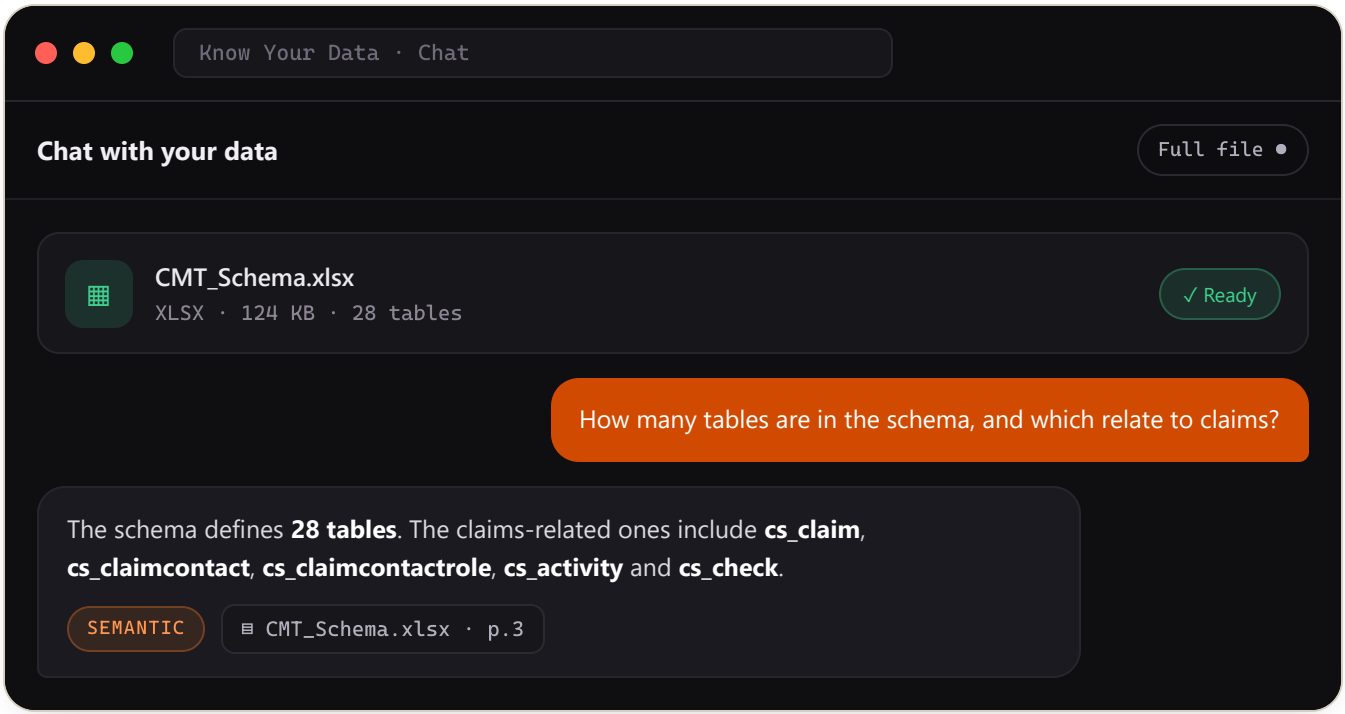
Three of the screens that carry the workflow, from asking questions of your own documents to generating deployable code.

KNOW YOUR DATA

## Chat with your insurance documents.

Upload policies, claims, DDL or spreadsheets. The app validates that a file is insurance-relevant, ingests it, and lets analysts ask questions in plain language — every answer cites the exact source it came from.

- Retrieval (RAG) with a whole-document mode for smaller files
- Answers routed to SQL or document search automatically
- Per-tenant & insurance-scoped — nothing leaks across clients

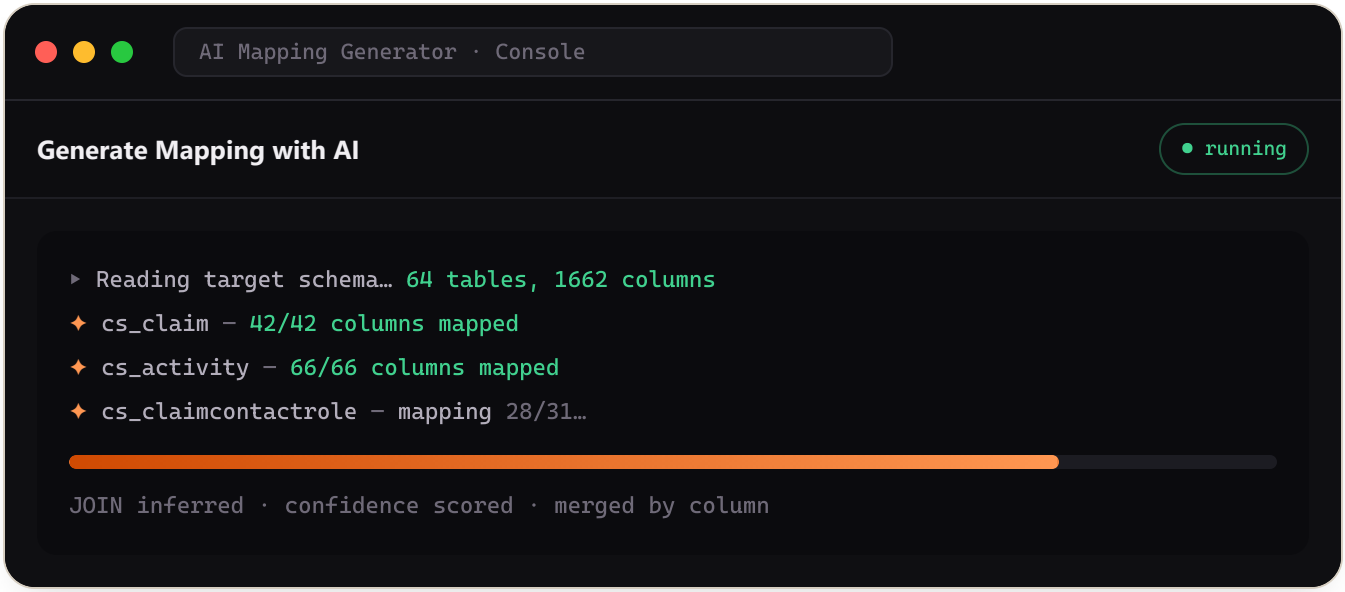


AI MAPPING GENERATOR

# Generate mappings table-by-table.

Pick target tables, tune the strategy and the editable prompt, then watch a live console generate mappings entity-by-entity — chunked so even the widest tables never truncate.

- Loops per entity and merges by column — no dropped fields
- Grounded on the real source; the model never invents tables
- Every run's token usage is tracked per feature and client

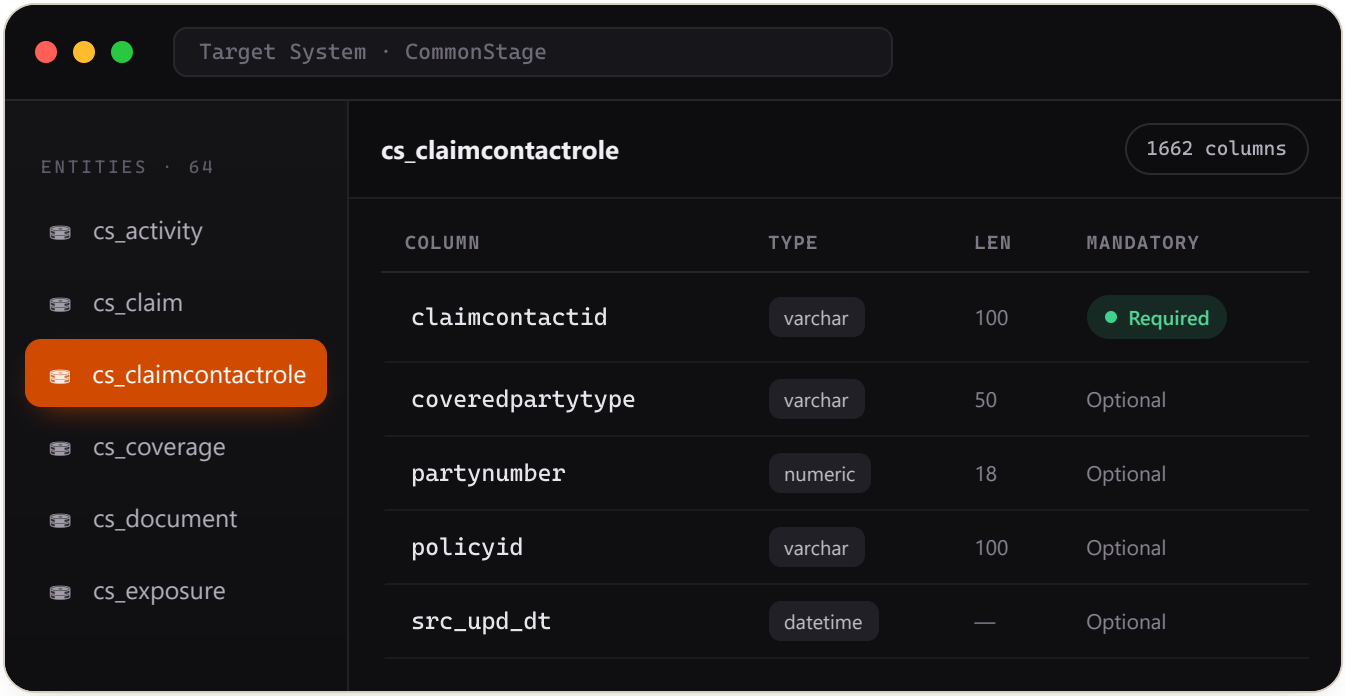


TARGET SYSTEM

# One active schema drives everything.

Manage SQL Server or file-based targets, browse the entity tree and every column's type, keys and defaults — the one active target schema feeds mapping, validation and CREATE TABLE generation.

- Live metadata read, or AI-extracted from an uploaded file
- Add, edit and delete tables & columns inline
- Schema defaults flow straight into generated DDL



HOW AI IS USED

# Claude, applied narrowly and safely.

Thirteen bounded operations — each returns structured data, SQL, or a grounded answer that the application parses and a person reviews. The model never acts on its own.

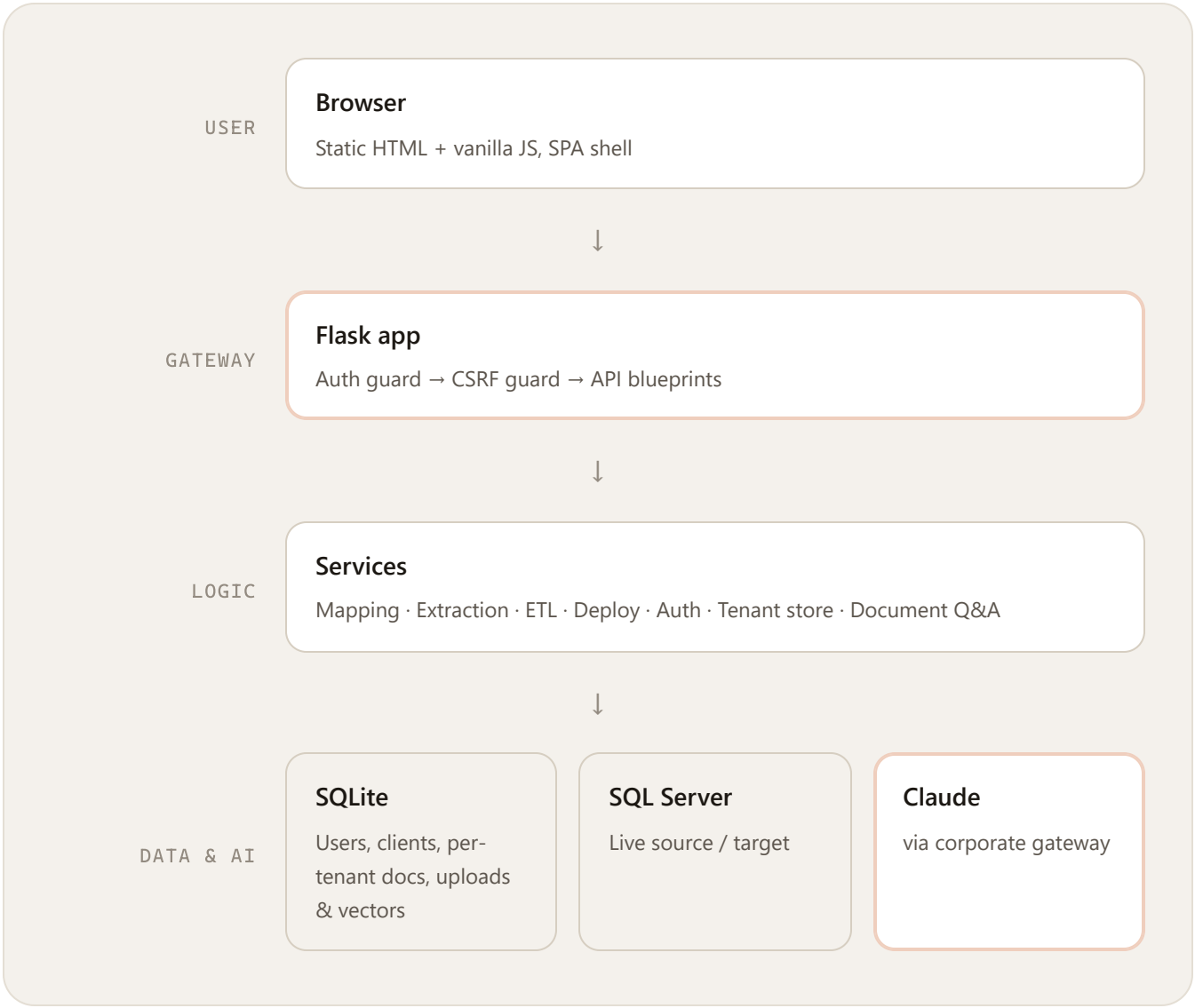
- **Mapping generation** — per target table
- **Schema extraction** — from uploaded files
- **ETL stored-procedure** generation
- **Add column / entity** from plain language
- **Insurance-domain check** — flags off-topic uploads
- **Text-to-SQL** — over ingested structured data
- **Single-field regeneration** — grounded on the real source
- **Rich extraction** — infers keys & nullability
- **CREATE TABLE DDL** generation
- **Deploy-time SQL auto-fix** — human-gated
- **Query routing** — structured vs. document questions
- **Grounded answers** — with source citations

**Guardrails.** Prompts forbid inventing tables or columns; hallucinated DDL is flagged; a deploy that fails is auto-corrected but **never auto-deployed** — the fix is surfaced for review. Know Your Data uses retrieval (RAG) over a local vector store and only answers from the user's own uploaded, insurance-relevant documents — every answer carries citations. Usage logs record token counts only — never prompt or response content, and **no autonomous agents** act on their own.

ARCHITECTURE

# One service, cleanly layered.

A static browser frontend and the API are served from a single origin. Everything privileged — database access, AI calls, persistence — happens on the server, scoped to the signed-in user and their active client.



Frontend: HTML + vanilla JavaScript, no framework, no build step. Backend: Flask, layered `api → services → parsers/schemas → core`. SQLite files hold the app's own data — including uploaded documents and their retrieval vectors; live SQL Server is only the conversion's source and target.

---

TRUST & SECURITY

# What's in place — and what's next.

An honest posture: strong tenant isolation and API hardening today; production-hardening still ahead. Nothing below is claimed unless it's in the code.

### In place `Implemented`

- ✓ Email/password auth, hashed passwords, admin-managed accounts (self-signup off by default)
- ✓ Per-tenant data isolation scoped to user + client, enforced server-side
- ✓ CSRF protection, login throttling, and connection-attempt rate limiting
- ✓ Connection passwords never persisted; no credentials in logs or AI prompts
- ✓ Human-gated deploys; all-or-nothing transactions with rollback
- ✓ Uploaded documents are per-tenant and insurance-scoped; chat answers stay grounded in them with citations

### Recommended next `Roadmap`

- → Run behind a production web server with debug off and HTTPS (currently a dev server)
- → Add CI, containerization, and automated frontend tests
- → Enforce upload size limits and pin dependency versions
- → Move to Postgres & shared storage if concurrency grows (schema is already portable)

WHERE IT STANDS

# Working software, documented.

- Core workflow functional
- Backend test suite (32 files)
- Full technical docs in `docs/`
- Not yet production-hardened

The codebase is clean and layered, with the security-critical paths (auth, tenant isolation, CSRF, deploy) covered by tests. The remaining work is concentrated in deployment readiness and frontend test coverage — not in core correctness.

Prepared from a full source-code analysis of AI Mapping Studio. The complete 28-document technical reference — architecture, API, prompt inventory, database, security, testing, and a developer guide — lives in the repository under `docs/`, indexed by `docs/00-README.md`.

This brief summarizes an internal engineering documentation set. Figures (operations, endpoints, tests) reflect the code as analyzed. The product screens shown are faithful, representative UI mockups built from the app's real theme and data — not captured images. No secrets or credentials are included.